Exp: 8 Write a C program to find FOLLOW( ) - predictive parser for the given grammar

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int limit, x = 0;
char production[10][10], array[10];

void find_first(char ch);
void find_follow(char ch);
void Array_Manipulation(char ch);

int main() {
   int count;
   char option, ch;

   printf("\nEnter Total Number of Productions:\t");
   scanf("%d", &limit);

   for (count = 0; count < limit; count++) {
      printf("\nValue of Production Number [%d]:\t", count + 1);
      scanf("%s", production[count]);
   }

   do {
      x = 0;
      printf("\nEnter production Value to Find Follow:\t");
      scanf(" %c", &ch);

      find_follow(ch);
```

```
        printf("\nFollow Value of %c:\t{ ", ch);
        for (count = 0; count < x; count++) {
            printf("%c ", array[count]);
        }
        printf("}\n");

        printf("To Continue, Press Y:\t");
        scanf(" %c", &option);

    } while (option == 'y' || option == 'Y');

    return 0;
}

void find_follow(char ch) {
    int i, j;

    // Rule 1: If ch is the start symbol, add '$' to its FOLLOW set
    if (production[0][0] == ch) {
        Array_Manipulation('$');
    }

    for (i = 0; i < limit; i++) {
        int length = strlen(production[i]);
        for (j = 2; j < length; j++) {
            if (production[i][j] == ch) {
                // Rule 2: If there is something after ch, find FIRST of that element
                if (production[i][j + 1] != '\0') {
                    find_first(production[i][j + 1]);
                }
```

```
            // Rule 3: If ch is at the end of the production, or the following part derives epsilon
            if (production[i][j + 1] == '\0' && ch != production[i][0]) {
                find_follow(production[i][0]);
            }
        }
    }
}

void find_first(char ch) {
    int k;

    // If ch is a terminal, add it directly
    if (!isupper(ch)) {
        Array_Manipulation(ch);
        return;
    }

    for (k = 0; k < limit; k++) {
        if (production[k][0] == ch) {
            // If production yields epsilon '$'
            if (production[k][2] == '$') {
                // If it derives epsilon, we might need to look at FOLLOW of LHS (handled via flow context, simplified here)
                // Keeping original structure while ensuring safety checks
                continue;
            } else if (islower(production[k][2])) {
                Array_Manipulation(production[k][2]);
            } else {
                find_first(production[k][2]);
            }
```

```
        }
    }
}

void Array_Manipulation(char ch) {
    int count;
    for (count = 0; count < x; count++) {
        if (array[count] == ch) {
            return;
        }
    }
    array[x++] = ch;
}
```

Output:

```
Enter Total Number of Productions:      4

Value of Production Number [1]: S=AaAb

Value of Production Number [2]: S=BbBa

Value of Production Number [3]: $

Value of Production Number [4]: $

Enter production Value to Find Follow:  S

Follow Value of S:      { $ }
To Continue, Press Y:   Y

Enter production Value to Find Follow:  A

Follow Value of A:      { a b }
To Continue, Press Y:   Y

Enter production Value to Find Follow:  B

Follow Value of B:      { b a }
To Continue, Press Y:   N


...Program finished with exit code 0
Press ENTER to exit console.
```